

# Friday Agent

本报告梳理 Friday Agent 的测评体系，覆盖  
pytest 单元测试、RAG 检索评测与 Agent 行  
为评测三层流程，以及语法检查、结构化日志与  
OTel 追踪等辅助手段，并附各环节的方法、指  
标、所用工具与最近一次实测结果。

---

整理自 EVALUATION.md · 2026 年 10 月

# 目 录

- 一、测评体系总览 .....3
- 二、单元测试 (pytest) .....3
  - 2.1 运行方式 .....3
  - 2.2 覆盖范围 .....3
  - 2.3 编写约定 .....4
  - 2.4 明确不测的部分（以手动验证为准） .....4
- 三、RAG 检索评测 (eval\_rag.py) .....4
  - 3.1 目的与被测链路 .....4
  - 3.2 评测流程 .....5
  - 3.3 指标定义 .....5
  - 3.4 所用工具与依赖 .....5
  - 3.5 金标设计原则（踩坑记录） .....6
  - 3.6 最近一次实测结果（2026-09-29） .....6
- 四、Agent 行为评测 (eval\_agent.py) .....6
  - 4.1 目的与用例集 .....6
  - 4.2 评测流程 .....6
  - 4.3 所用工具与依赖 .....7
  - 4.4 最近一次实测结果 .....7
- 五、辅助质量保障 .....7
- 六、已知问题与演进方向 .....8

## 一、测评体系总览

Friday Agent 项目采用「三层测评 + 辅助检查」的质量保障结构：三层测评分别覆盖纯逻辑单元、RAG 检索链路与 Agent 端到端行为，被测对象逐层升级、外部依赖逐层引入；辅助检查负责静态正确性与运行期可观测性。各层的定位、位置与依赖如下表所示。

|                       |                       |                       |               |
|-----------------------|-----------------------|-----------------------|---------------|
| 1.0<br>RAG 检索评测 Hit@4 | 0.944<br>RAG 检索评测 MRR | 3 / 3<br>Agent 行为评测通过 | 7<br>单元测试覆盖模块 |
|-----------------------|-----------------------|-----------------------|---------------|

| 层级 | 名称              | 位置                                | 被测对象                                | 外部依赖                   | 建议频率            |
|----|-----------------|-----------------------------------|-------------------------------------|------------------------|-----------------|
| L1 | 单元测试            | backend/tests/<br>(pytest)        | 上下文装配、记忆、权限、日志中间件等纯逻辑               | 无（不触网、不落库）             | 每次改动后           |
| L2 | RAG 检索评测        | backend/scripts/<br>eval_rag.py   | ES 混合检索链路（稠密 + BM25 + RRF + rerank） | ES、智谱 embedding、rerank | 语料、索引、检索参数变更后   |
| L3 | Agent 行为评测      | backend/scripts/<br>eval_agent.py | Agent 端到端工具决策与回答质量                  | LLM API、ES             | 模型、Prompt、工具变更后 |
| 辅助 | 语法 / 类型检查       | compileall /<br>tsc --noEmit      | 前后端代码静态正确性                          | 无                      | 每次改动后           |
| 辅助 | 结构化日志 + OTel 追踪 | logging_config.py<br>tracing.py   | 运行时可观测性（非 pass / fail 断言）           | OTLP 导出（可选）            | 运行期持续           |

表 1 测评体系分层总览

## 二、单元测试 (pytest)

### 2.1 运行方式

```
cd backend
.venv/bin/python -m pip install -r requirements-dev.txt # 开发依赖: pytest>=8.0
.venv/bin/python -m pytest tests/ -q
```

### 2.2 覆盖范围

共 7 个测试文件，覆盖后端全部核心纯逻辑模块；最近一次全量运行全部通过。

| 测试文件                  | 被测模块                 | 覆盖内容                                                                         |
|-----------------------|----------------------|------------------------------------------------------------------------------|
| test_context.py       | app/context.py       | token 近似估算（中文 ÷ 1.6 / ASCII ÷ 4）、预算窗口切分、游标排除、消息清洗（图片占位、空消息丢弃）                |
| test_memory.py        | app/memory.py        | 长期记忆行级去重（空白归一化、保序）、抽取触发计数（游标定位、游标失效自愈、游标在末尾）                                 |
| test_agent_logging.py | app/agent_logging.py | AgentLoggingMiddleware 钩子透传与节点日志断言；含用假模型走真实 Agent 装配的集成冒烟                    |
| test_agent_tools.py   | app/agent_tools.py   | 内置工具权限边界：deny 规则覆盖面（.env / .db / .git / .ssh / 源码目录）、工作区唯一豁免、DONT_ASK 模式引擎裁决 |
| test_confirm.py       | app/confirm.py       | 确认协调器 ConfirmHub：登记 / 应答唤醒、归属校验（非本人不能应答）、超时默认拒绝、取消幂等                         |
| test_conversations.py | app/conversations.py | 会话列表分页 / 搜索纯函数：LIKE 通配符转义（% / _ / \ 按字面量匹配）、页大小约定（默认 20）                     |
| test_workspace.py     | workspace_picker 等   | 工作区根目录校验（相对路径拒绝、敏感目录拒绝、文件非目录拒绝）、会话设置 schema（权限模式枚举、工作区选定后不可置空）               |

表 2 单元测试覆盖范围（7 个测试文件）

2.3 编写约定

全部单测遵循同一套约定，保证不触网、不落库、不依赖 DB / HTTP 基建，可在任何环境秒级跑完：

- 外部对象用 SimpleNameSpace 假件模拟，不引入 mock 框架
- 异步逻辑用 asyncio.run 包成同步用例
- 文件系统用 pytest 的 tmp\_path fixture
- 开发者本机 .env 用 monkeypatch 隔离，权限裁决用例默认无 Bash 放行前缀

2.4 明确不测的部分（以手动验证为准）

- 分页与过滤的 SQL 行为：需要 DB / HTTP 测试基建（项目现无测试数据库），以手动验证为准
- 前端暂无自动化测试（无 \*.test.\* / \*.spec.\* 文件），以 tsc --noEmit 类型检查与生产构建兜底

三、RAG 检索评测 (eval\_rag.py)

3.1 目的与被测链路

验证医学文献混合检索链路的召回与排序质量。被测链路即生产环境真实检索路径 rag\_search，不做任何 mock：

```
query → 智谱 embedding-3 向量化 (1024 维)
      → ES 双路召回: 稠密 knn + BM25 (IK 中文分词)
      → 应用层 RRF 融合: score =  $\sum 1/(60 + \text{rank})$ 
      → 智谱 rerank 精排 → 返回 top_k
```

两点设计说明: ES 服务端 RRF retriever 是付费许可功能 (Basic license 返回 403), 故在客户端按秩融合, 数学上与服务端 RRF 等价; rerank 调用失败时自动降级为 RRF 排序, 属于预期兜底而非错误。

### 3.2 评测流程

- 遍历 9 条 golden 用例 (8 条中文 + 1 条英文跨语言探测: measles vaccination → 期望命中「麻疹」)
- 对每条调用真实的 rag\_search(query, top\_k=4)
- 在返回标题中 (不区分大小写) 查找期望关键词首次出现的位置 rank
- 计算 Hit@4 / MRR 指标, 逐条打印检索明细 (得分 + 标题 + 命中标记)
- (可选) 对每个命中 chunk 调用 LLM 裁判做 0 - 2 分相关性评分
- 汇总并落盘 JSON 报告至 backend/logs/eval\_rag.json

### 3.3 指标定义

| 指标            | 定义                  | 说明                          |
|---------------|---------------------|-----------------------------|
| Hit@4         | top-4 内出现期望关键词的用例占比 | 主指标                         |
| MRR           | 期望结果首次出现排名的倒数的平均    | 排序质量                        |
| avg_top_score | top-1 rerank 分均值    | 参考项: rerank 分数饱和, 区分度有限     |
| judge_avg     | LLM 裁判 0 - 2 分均值    | 可选项, 需配置 EVAL_JUDGE_API_KEY |

表 3 RAG 评测指标

### 3.4 所用工具与依赖

- 被测系统: Elasticsearch 9.x (med-es 容器, 自签 TLS + basic auth)、IK 中文分词插件、智谱 embedding-3 (稠密路)、智谱 rerank (精排, 失败自动降级为 RRF 序)
- LLM 裁判 (可选): httpx 直调智谱 OpenAI 兼容接口, 默认模型 glm-4-flash; 由环境变量控制, 即 EVAL\_JUDGE\_API\_KEY (未配置自动跳过)、EVAL\_JUDGE\_BASE\_URL 与 EVAL\_JUDGE\_MODEL
- 输出: 终端逐条明细 + logs/eval\_rag.json 结构化报告 + 「评测[RAG\*]」结构化日志

### 3.5 金标设计原则（踩坑记录）

- expect 关键词必须与语料实际标题（中文）匹配：旧英文关键词集（opioid / parkinson）对中文标题语料永远 MISS，属假阴性
- 选例前需先确认语料主题覆盖（语料中无糖尿病、帕金森等主题文档）

### 3.6 最近一次实测结果（2026-09-29）

|                       |              |                  |               |
|-----------------------|--------------|------------------|---------------|
| 1.0<br>Hit@4（9/9 全命中） | 0.944<br>MRR | 9 条<br>golden 用例 | 5.33 s<br>总耗时 |
|-----------------------|--------------|------------------|---------------|

唯一非 rank-1 用例是「面瘫有哪些症状」：期望关键词「面瘫」排第 2，第 1 为同义标题「面神经麻痹」，属可接受偏差。英文跨语言用例 measles vaccination 命中「麻疹」，rank 1。LLM 裁判未启用（key 留空，裁判逻辑已跑通，配置即用）。

## 四、Agent 行为评测（eval\_agent.py）

### 4.1 目的与用例集

端到端验证 Agent 的工具决策与回答质量，使用真实模型与真实检索、不做任何 mock，共 3 条用例：

| 用例                | 问题              | 期望                                              |
|-------------------|-----------------|-------------------------------------------------|
| 医学问题应触发 RAG 工具    | 帕金森病的主要症状是什么？   | 调用 medical_rag_search 工具，回答含帕金森 / parkinson 关键词 |
| 跨语言医学问题应触发 RAG 工具 | 丙型肝炎有哪些症状和传播途径？ | 调用工具，回答含「肝 / hepatitis」                         |
| 非医学问题不应触发工具       | 用一句话介绍你自己       | 不调用任何工具，回答含「friday / 助手 / 智能」                   |

表 4 Agent 行为评测用例集

### 4.2 评测流程

1. 构造 AgentService（关闭 thinking 与 web\_search，取纯基线）
2. 逐用例消费 agent.reply\_stream() 原生事件流，用 agentscope 内置 ConsoleRenderer 做终端可视化：  
--verbose 逐事件渲染文本增量、思考、工具调用与工具结果，默认 quiet 模式只出结论
3. 从事件流收集工具调用名（TOOL\_CALL\_START）与回答文本（TEXT\_BLOCK\_DELTA）
4. 判定：工具决策符合预期（该调 / 不该调）且回答含任一期望关键词即为 PASS

5. （可选）LLM 裁判对回答打 1 – 5 分（相关性、完整性），配置同 RAG 评测的 EVAL\_JUDGE\_\* 环境变量
6. 汇总落盘 logs/eval\_agent.json

4.3 所用工具与依赖

- agentscope 2.0.8: ConsoleRenderer（运行轨迹可视化）、事件流 EventType、UserMsg
- 被测 Agent: app/agent.py 的 AgentService（需配置 MODEL\_PROVIDER=deepseek 与 OPENAI\_API\_KEY）
- LLM 裁判（可选）：httpx 调智谱 glm-4-flash

4.4 最近一次实测结果

|                 |                |                    |                   |
|-----------------|----------------|--------------------|-------------------|
| 3 / 3<br>用例全部通过 | 16.54 s<br>总耗时 | 2 条<br>医学用例均正确触发工具 | 1 条<br>非医学用例零工具调用 |
|-----------------|----------------|--------------------|-------------------|

两条医学用例的回答长度分别为 1161 / 1652 字符，均含期望关键词；非医学用例未触发任何工具且回答含「friday / 助手 / 智能」。LLM 裁判未启用（key 留空）。

五、辅助质量保障

| 手段               | 命令 / 入口                                       | 说明                                  |
|------------------|-----------------------------------------------|-------------------------------------|
| 后端语法检查           | python -m compileall -q app                   | 快速静态校验                              |
| 前端类型检查           | npx tsc --noEmit                              | TypeScript 编译期检查（前端无单测，以此兜底）        |
| 前端构建             | npm run build                                 | 生产构建验证                              |
| 结构化日志            | backend/logs/backend.log (DEBUG+, 5MB × 3 轮转) | 「评测[...]」 「节点[RAG*]」等关键节点日志，评测与排障共用 |
| OpenTelemetry 追踪 | app/tracing.py (OTLP 导出，可接 Langfuse 等)        | 调用链观测；未开启时中间件自动短路，近零开销              |
| 孤儿文件清理           | scripts/cleanup_files.py [--delete]           | 运维脚本（非测评）：按消息引用关系安全清理未引用图片，默认只预览    |

表 5 辅助质量保障手段

## 六、已知问题与演进方向

---

1. 语料向量与查询向量模型是否同源存疑（历史失配，稠密路区分度受损），当前由 BM25 稀疏路 + rerank 兜底补足（混合检索下 Hit@4 = 1.0）；根治方案为用 embedding-3 重建语料向量，eval\_rag.py 可量化验证重建效果
2. rerank 分数饱和（普遍接近 1.0），top\_score 指标参考意义下降，无法做绝对阈值过滤；如需可阈值化分数，建议换打分更分散的 rerank 模型（如 BAAI/bge-reranker-v2-m3）
3. LLM 裁判尚未启用：两个评测脚本的裁判逻辑均已实现且可跑通，但 EVAL\_JUDGE\_API\_KEY 暂留空，配置后即可获得相关性（0 - 2）与回答质量（1 - 5）评分维度
4. 无 CI、无集成测试：分页 / 搜索的 SQL 行为、鉴权、SSE 主链路依赖手动验证；演进方向为补齐鉴权 / 会话 / SSE 三条主链路的自动化测试
5. golden 用例规模小（RAG 9 条、Agent 3 条），适合快速回归；扩大语料覆盖后应同步扩充用例集